Nguyễn Anh Đức - N25DECE079 - E25CQCE02-N

Problem 1: Singly Linked List - In-place Reversal & Pointer Tracing

Given a singly linked list: head → [10] → [25] → [40] → [15] → ∅

- Task 1: Write a function / algorithm in pseudocode to reverse the singly linked list, including three pointers (prev, curr, next) to complete your function.
- Task 2: Draw a diagram or step-by-step pointer trace table showing the positions ~~the~~ and values of the three pointers (prev, curr, next) after each iteration until the list is fully reversed.

Note: Do not use algorithms library

Answer

Task 1: Pseudocode

```
function reverseLinkedList(head)
    prev = NULL
    curr = head
    next = NULL
    while (curr is not NULL)
        next = curr.next
        curr.next = prev
        prev = curr
        curr = next
    head = prev
    return head
```

BÚT CẦU

Scanned with CamScanner

1,25 Task 2: Pointer trace table

Iteration	prev	curr	next	statement
Start	NULL	10	Ø	10 → 25 → 40 → 15 → NULL
After round 1	10	25	25	10 → NULL
After round 2	25	40	40	25 → 10 → NULL
After round 3	40	15	15	40 → 25 → 10 → NULL
After round 4	15	NULL	NULL	15 → 40 → 25 → 10 → NULL

End: Because curr == NULL, Assign head = prev (Node 15)

New list: Head → [15] → [40] → [25] → [10] → NULL

2.5 Problem 2: Circular linked list - Insert at head

Given a singly circular linked list: head → [10] → [20] → [30] → to head

Task 1: Write pseudocode for the function InsertAtHead(head, newValue) to insert a new node containing newValue at the front of the circular linked list while maintaining its circular structure.

Task 2: Trace the execution steps (via text description or pointer diagram) when inserting newValue = 5 into the given list.

Answer

1,25 Task 1: Pseudocode

```
function InsertAtHead(head, newValue)
    newNode = createNode(newValue)
    if head is NULL:
        newNode.next = newNode
        return newNode
```

HL BÚT CẤU

Scanned with CamScanner

tail = head
while (tail.next != head)
    tail = tail.next
newNode.next = head
tail.next = newNode
head = newNode
return head

1.25 Task 2: Trace with newValue = 5

Initial list: [10] → [20] → [30] → (10)

Create a new node: newNode contain the value 5. After that, traverse through nodes 10 → 20 → 30. Notice that 30.next == head, stop with tail = 30

Update pointer:
- newNode.next = head → 5.next points to 10
- tail.next = newNode → 30.next points to 5
- head = newNode → new head is node 5

=> Result: Head → [5] → [10] → [20] → [30] → (points back to 5)

2.5 Problem 3: Doubly linked list - Insert before Target & Edge cases

Given a doubly linked list containing integers.

Task 1: Write pseudocode for InsertBefore(head, target, newValue) to insert a new node containing newValue immediately before the first occurrence of target.

Task 2: List the exact order of pointer assignments required and briefly justify why this specific sequence avoids memory leaks or broken lists

Task 3: ~~Explicitly~~ Explicitly handle the following two edge case in code:
- Target is located at the head Node (head)
- Target does not exist in the list

Answer

BÚT CẦU

Scanned with CamScanner

Task 3

1.75 Task 1: Pseudocode

```
function InsertBefore (head, target, newValue)
    if if head is NULL: return NULL // Empty list
    curr = head
    While (curr is not NULL and curr.data != target)
        curr = curr.next
    if curr is NULL:
        print ("Target not found")
        return head
    newNode = create Node (newValue)
    if curr == head:
        newNode.next = head
        head.prev = newNode
        head = newNode
        return head
    newNode.prev = curr.prev
    newNode.next = curr
    curr.prev.next = newNode
    curr.prev = newNode
    return head
```

HL BÚT CẦU

Scanned with CamScanner

0,75 Task 2: Pointer Assignment order and Explanation

Pointer assignment order for inserting into the middle (of a Doubly Linked list)

- ① newNode.prev = curr.prev
- newNode.next = curr
- ② curr.prev.next = newNode
- ③ curr.prev = newNode

Explanation:

- Establish the two outgoing links from newNode first to connect the new node to the chain without losing track/losing the addresses of the surrounding nodes
- Next, update the next pointer of the preceding node to point to newNode
- Finally, overwrite curr.prev to point to newNode. If we modify curr.prev first, we will lose the reference to the node ~~pred~~ preceding curr, leading to broken links and memory leaks

1,25 Problem 4:

Given a singly linked list sorted in ASC order:

Answer

```
function InsertInOrder(head, newValue)
    newNode = createNode(newValue)
    if (head is NULL or newValue < head.data)
        ① newNode.next = head
        head = newNode
        return head
```

Tiến học tế - Học tế tiến

Quang Tâm

Scanned with CamScanner

```
curr = head
while(curr.next is not NULL and curr.next.data < newValue)
    curr = curr.next
newNode.next = curr.next
curr.next = newNode
return head.
```


Quang Tâm

Scanned with CamScanner